



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Procesamiento de Lenguaje Natural

github.com/marcelomendoza/IIC3670

Marcelo Mendoza

La tabla no necesita una fila por palabra

La clase 4 armó el vector de una palabra desde sus letras. Lo general no es el alfabeto sino que el conjunto de símbolos que indexa la tabla se elige.

$$W_{in} \in \mathbb{R}^{|S| \times d}$$

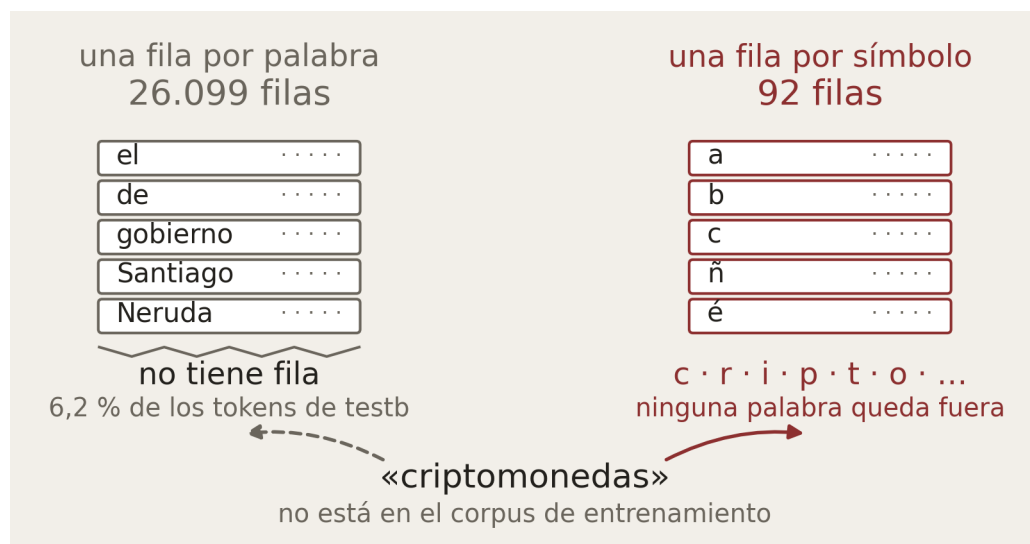
una fila por símbolo de S , no por palabra

$$S = \mathcal{V} : |S| = 26.099$$

las palabras vistas; $w \notin \mathcal{V}$ no tiene fila

$$S = \mathcal{A} : |S| = 92$$

el alfabeto; toda palabra se escribe con $\mathcal{A}$



Con palabras siempre falta alguna

26.099 filas, y el 6,2 % de los tokens de testb no tiene ninguna.

Con el alfabeto no falta nada

92 símbolos, y toda palabra del español se escribe con ellos.

conll2002, esp.train para la tabla y esp.testb para la medición.

Byte Pair Encoding fusiona el par más frecuente

Nace como un algoritmo de compresión en 1994. Acá no comprime bytes sino que fusiona símbolos hasta que el vocabulario llega al tamaño pedido.

$$S_0 = \mathcal{A}$$

se parte del alfabeto y nada más

$$(a, b)_t = \arg \max_{(a,b)} c_t(a, b)$$

el par adyacente más frecuente del corpus

$$S_t = S_{t-1} \cup \{ab\}, \quad t = 1, \dots, k$$

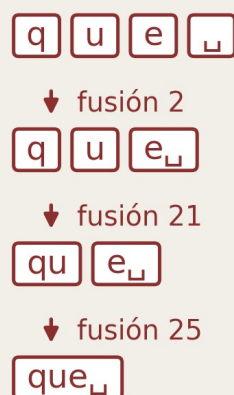
la fusión entra al vocabulario

las primeras fusiones sobre el corpus

1	a + $\square$	→	a $\square$	42.858
2	e + $\square$	→	e $\square$	39.616
6	d + e $\square$	→	de $\square$	18.837
8	e + n	→	en	14.889
21	q + u	→	qu	9.529
25	qu + e $\square$	→	que $\square$	8.021

$\square$ marca el fin de palabra

y así se arma «que»



Nadie escribe reglas de morfología

Se cuenta sobre el corpus entero y gana la frecuencia.

Cada fusión se apoya en las anteriores

«que» junta dos piezas que al principio no existían.

Segmentar es repetir las fusiones

En el orden en que se aprendieron, no buscando la pieza más larga.

Cada fusión trae su propia fila

El árbol de fusiones decide cómo se corta la palabra y ahí termina su trabajo. Los vectores no se heredan.

$$W_{in} \in \mathbb{R}(|\mathcal{A}|+k) \times d$$

una fila por letra y una por fusión

$$v_{s_j} = W_{in}^\top x_{s_j}$$

la pieza busca su fila, como una palabra

$$v_{ab} \neq g(v_a, v_b)$$

la fusión ab no compone: su fila es libre

la tabla de embeddings

a	b	c	...	ñ
:	:	:	:	:

una fila por letra
92 filas

a_	qu	que_
:	:	:

una fila por fusión
4.000 filas

4.092 filas en total

y ninguna palabra sin representación

cada pieza busca su propia fila

«que»

que_

una pieza, un vector

«queso»

qu

dos piezas, dos vectores

eso_

«que_» y «qu» son dos filas

una no sale de la otra
cada fusión, su propio vector

Hay un embedding por fusión

El alfabeto pone 92 filas y las fusiones 4.000, o sea 4.092 en total.

Pero la fusión no compone nada

El vector de «que» no sale del de «qu». Se aprende solo.

No es lo que hacían los subwords

Allá la palabra era la suma de sus piezas. Acá la compone la red.

Vocabulario y segmentaciones de BPE entrenado sobre conll2002 esp.train.

Entre la letra y la palabra

Los dos diseños son los extremos de un mismo eje. El precio de un vocabulario chico es que la palabra se parte en más piezas.

$$w \longrightarrow (s_1, \dots, s_m), \quad s_j \in S$$

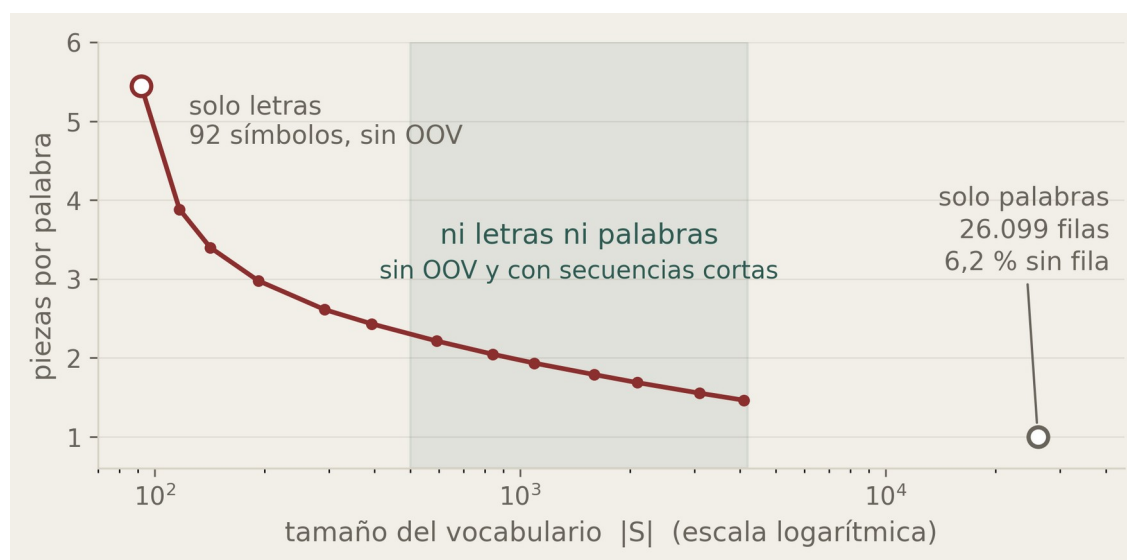
la palabra es una secuencia de piezas de S

$$m = 1 (\mathcal{V}), \quad m = 5,44 (\mathcal{A})$$

sobre testb: el alfabeto alarga la frase

$$|S| = |\mathcal{A}| + k$$

el alfabeto más k piezas aprendidas



El alfabeto alarga la frase

5,44 piezas por palabra, o sea una oración cinco veces más larga.

El vocabulario deja palabras afuera

26.099 filas y, aun así, palabras sin fila.

BPE no elige un extremo

Con 200 fusiones ya baja de 5,44 a 2,61 piezas por palabra.

Piezas por palabra medidas sobre conll2002 esp.testb, con BPE entrenado en esp.train.

Cada posición mezcla lo que hay hasta ella

Los coeficientes dicen cuánto toma una posición de cada una de las que llegan hasta ella. Y una posición es una pieza de BPE, no una palabra.

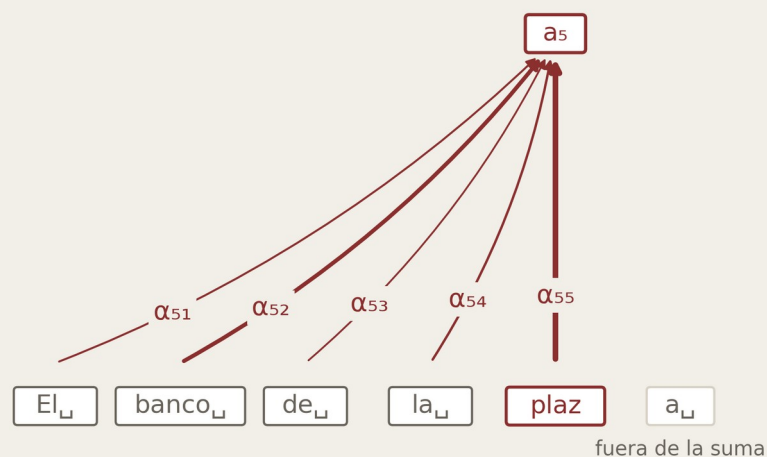
$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

mezcla las entradas hasta la posición i

$$\sum_{j \leq i} \alpha_{ij} = 1, \quad \alpha_{ij} \geq 0$$

los coeficientes reparten un total de uno

la salida de la posición 5 mezcla las 5 primeras entradas



los 5 coeficientes suman 1; lo que viene después de la posición 5 no entra

Un coeficiente por cada entrada

Dice cuánto toma la posición i de la posición j .

Reparten un total de uno

Si una entrada se lleva más, otra se lleva menos.

La suma llega hasta i

La posición 5 mezcla las cinco primeras; la sexta no entra.

Bahdanau, Cho y Bengio, ICLR 2015 (los coeficientes); Cheng, Dong y Lapata, EMNLP 2016 (sobre la misma secuencia).

Comparar, normalizar, mezclar

El puntaje es un producto punto, la softmax lo vuelve peso, y la salida es la mezcla de las entradas con esos pesos.

$$\text{score}(x_i, x_j) = x_i \cdot x_j$$

comparar es multiplicar

$$\alpha_{ij} = \text{softmax}_{j \leq i}(\text{score}(x_i, x_j))$$

los puntajes se vuelven pesos

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

la salida es la mezcla ponderada

la fila de «ado_┘»: su puntaje contra cada pieza hasta la suya

palabra	producto punto	peso tras la softmax
El	El _┘ 2,1	0,002
banco	banco _┘ 1,3	0,001
de	de _┘ 1,3	0,001
la	la _┘ 1,0	0,001
plaza	plaz _┘ 1,3	0,001
	a _┘ 1,3	0,001
estaba	estaba _┘ 1,9	0,001
	p _┘ 1,0	0,001
pintado	int _┘ 1,9	0,001
	ado _┘ 8,5	0,992

7 palabras ocupan 10 posiciones, y «ado_┘» se lleva 0,992 de su propio peso

Comparar es multiplicar

El puntaje de una pieza contra otra es su producto punto, nada más.

Las posiciones no son palabras

Siete palabras ocupan diez posiciones, y la consulta es un sufijo.

Así todavía no sirve

Sin parámetros, cada pieza se lleva 0,992 de su propio peso.

La misma pieza, en tres roles

Tres matrices aprendidas convierten cada entrada en consulta, clave y valor. Con eso se arregla lo que la lámina anterior dejó roto.

$$\mathbf{q}_i = W^Q x_i, \quad \mathbf{k}_i = W^K x_i, \quad \mathbf{v}_i = W^V x_i$$

el mismo x_i en tres roles

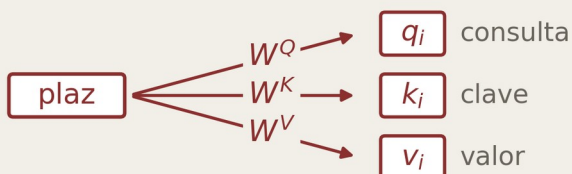
$$\text{score}(x_i, x_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

la consulta de i contra la clave de j

$$\text{head}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

se mezclan los valores, no las entradas

una entrada, tres roles



y entonces preguntar deja de ser recíproco

«plaz» pregunta por «banco_» $\mathbf{q}_i \cdot \mathbf{k}_j$

«banco_» pregunta por «plaz» $\mathbf{q}_j \cdot \mathbf{k}_i$

antes, con un solo vector por posición, eran el mismo producto

Consulta, clave y valor

El que pregunta, el que es preguntado y el que aporta contenido.

Preguntar deja de ser recíproco

Lo que i opina de j ya no tiene por qué ser lo que j opina de i .

Se mezclan los valores

Lo que se compara y lo que se suma dejan de ser lo mismo.

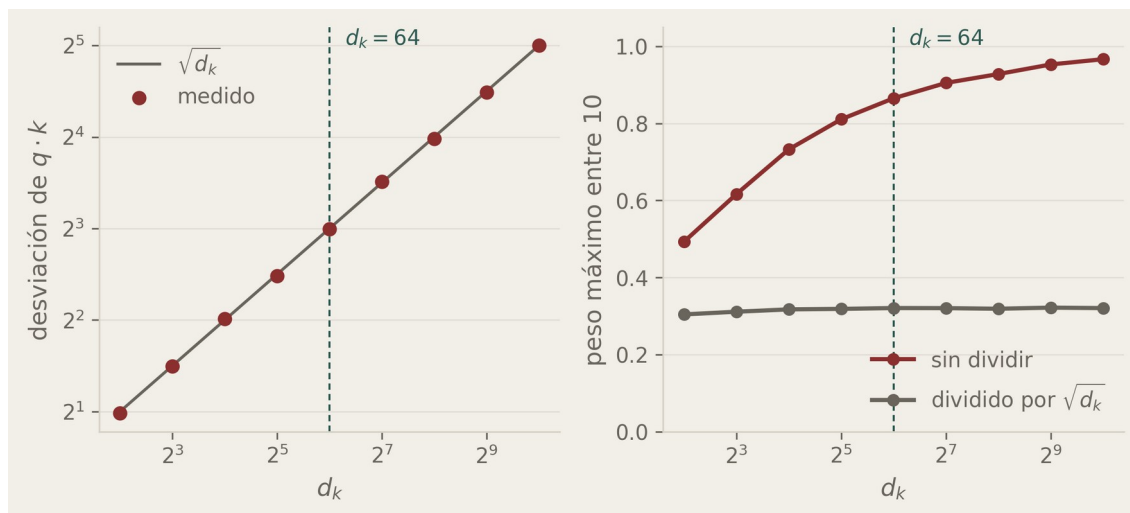
Tres matrices para todas las posiciones

Las tres son las mismas en toda posición y no dependen del largo de la frase. Y el divisor, la raíz de la dimensión, sale de la varianza.

W^Q, W^K, W^V : no dependen de i las mismas en toda posición

$\text{Var}(q \cdot k) = d_k$ si las componentes tienen varianza 1

$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$ devuelve la desviación a uno



Una sola copia, no una por posición

La misma pieza en dos lugares recibe exactamente la misma consulta.

Por eso el orden hay que inyectarlo

La posición se suma antes, desde una tabla aparte.

El divisor sale de la varianza

La varianza del producto punto es la dimensión; la raíz la devuelve a uno.

La atención ocurre en menos dimensiones

Se entra en 512 y cada cabezal compara y mezcla en 64. Los A cabezales se concatenan, y es la matriz de salida la que proyecta ese vector a 512.

$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c v_j^c$$

c es el cabezal, $1 \leq c \leq A$

$$W_c^Q, W_c^K, W_c^V \in \mathbb{R}^{64 \times 512}$$

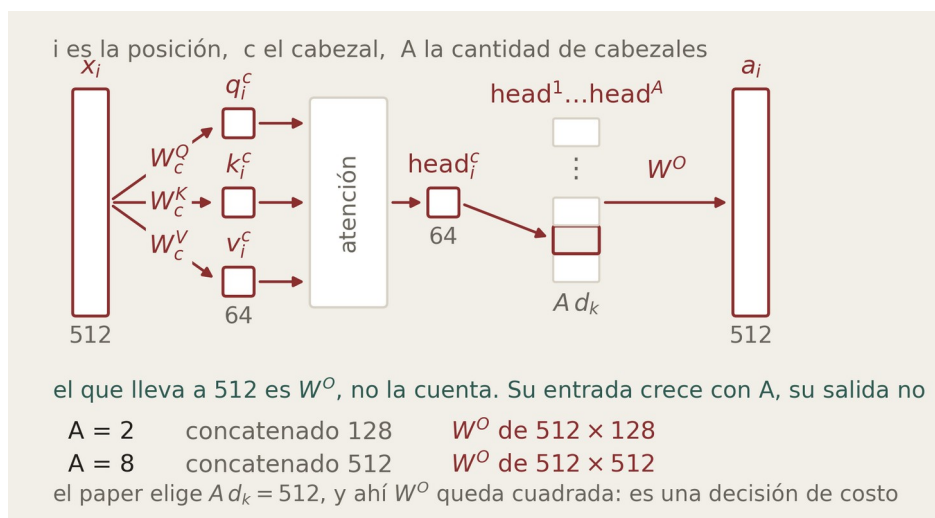
cada cabezal baja a 64

$$a_i = W^O(\text{head}_i^1 \oplus \dots \oplus \text{head}_i^A)$$

concatenados, no uno solo

$$W^O \in \mathbb{R}^{512 \times A d_k}$$

W^O proyecta a 512, sea cual sea A



La matriz de salida los ve a todos

Su entrada son los A cabezales pegados, no uno solo.

Las tres matrices no son cuadradas

Proyectan de 512 a 64, y la atención pasa por ese cuello.

A no está atado a la salida

Con dos cabezales entran 128 y con ocho 512; se sale en 512 igual.

La conexión residual

La salida de la atención no reemplaza a la entrada, se le suma. Ese atajo salva al gradiente de apagarse, y a cambio deja que la escala crezca.

$$h_\ell = \mathbf{h}_{\ell-1} + a(h_{\ell-1})$$

la conexión residual suma la entrada

$$\frac{\partial h_\ell}{\partial h_{\ell-1}} = \mathbf{I} + \frac{\partial a}{\partial h_{\ell-1}}$$

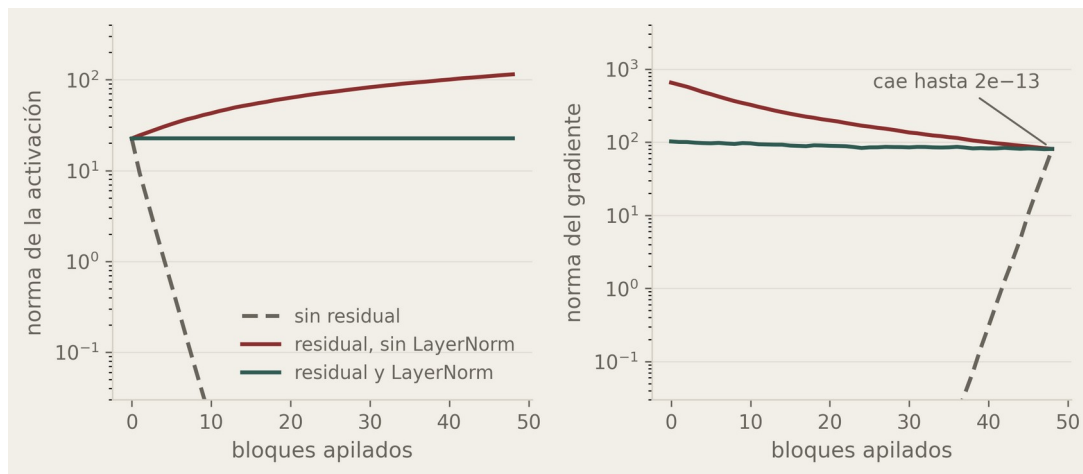
la identidad no atenúa el gradiente

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_L} \prod_{\ell=1}^L \frac{\partial h_\ell}{\partial h_{\ell-1}}$$

pero son L factores multiplicados

$$h_\ell = \text{LayerNorm}(h_{\ell-1} + a(h_{\ell-1}))$$

normalizar acota cada factor



Simulación: 48 bloques de dimensión 512, con y sin residual, con y sin LayerNorm.

Sumar la entrada deja un atajo

Al derivar aparece la identidad.

Sin él, en 48 bloques no queda nada

El gradiente llega a la entrada en cero, y las primeras capas no aprenden.

Pero la escala se acumula

Normalizar después de sumar la deja plana sin tocar el atajo.

LayerNorm

Se le resta la media y se lo divide por su desviación, calculadas sobre el propio vector. Después una ganancia y un sesgo que sí se aprenden.

$$\mu = \frac{1}{d} \sum_{r=1}^d x_r$$

la media de las d componentes

$$\sigma = \sqrt{\frac{1}{d} \sum_{r=1}^d (x_r - \mu)^2}$$

y su desviación, del mismo vector

$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sigma} + \beta$$

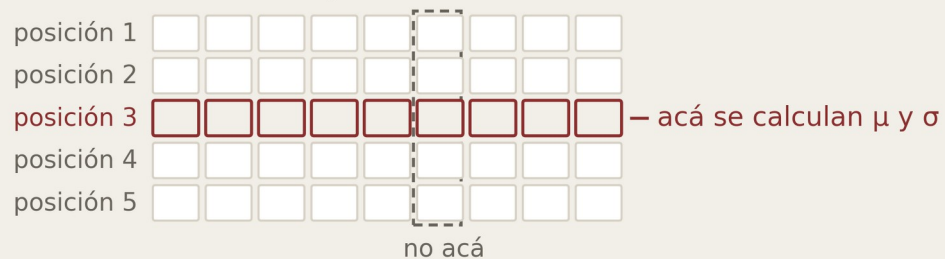
γ y β se aprenden

$$\text{LayerNorm}(\alpha x) = \text{LayerNorm}(x)$$

la escala de la entrada no pasa

se estandariza cada vector por separado

las d componentes del vector



nada cruza entre posiciones, así que la operación no depende del largo de la frase
un vector real de GPT-2: antes $\mu = 0,05$ $\sigma = 3,44$ · después $\mu = 0$ $\sigma = 1$
y como μ y σ salen del propio vector, multiplicarlo por una constante no cambia nada

La media es del propio vector

Sobre sus d componentes, no sobre el lote ni sobre la frase.

Gamma y beta se aprenden

La red decide con qué escala y en torno a qué centro quiere quedar.

La escala de la entrada no pasa

Multiplicar el vector por una constante deja la salida idéntica.

Después de mezclar, transformar

La atención mezcla pero no transforma, porque todo lo que le pasa a los valores es lineal. La no linealidad la pone una red que corre en cada posición.

$$\begin{aligned} a_i &= \text{MultiHead}(x_i, [x_1, \dots, x_j]), \quad j \leq i && \text{mezcla las posiciones} \\ z_i &= \text{LayerNorm}(x_i + a_i) && \text{residual, y después normalizar} \\ f_i &= W_2 \max(0, W_1 z_i) && \text{cada posición por su cuenta} \\ y_i &= \text{LayerNorm}(z_i + f_i) && \text{residual y normalizar otra vez} \end{aligned}$$



La atención no toca el contenido

La softmax decide los pesos; los valores solo se proyectan y se suman.

La red mira una posición a la vez

La mezcla ya la hizo la atención; acá no se cruza nada.

Dos sublayers, la misma receta

Cada uno con su residual y su LayerNorm, y el ancho vuelve a 512.

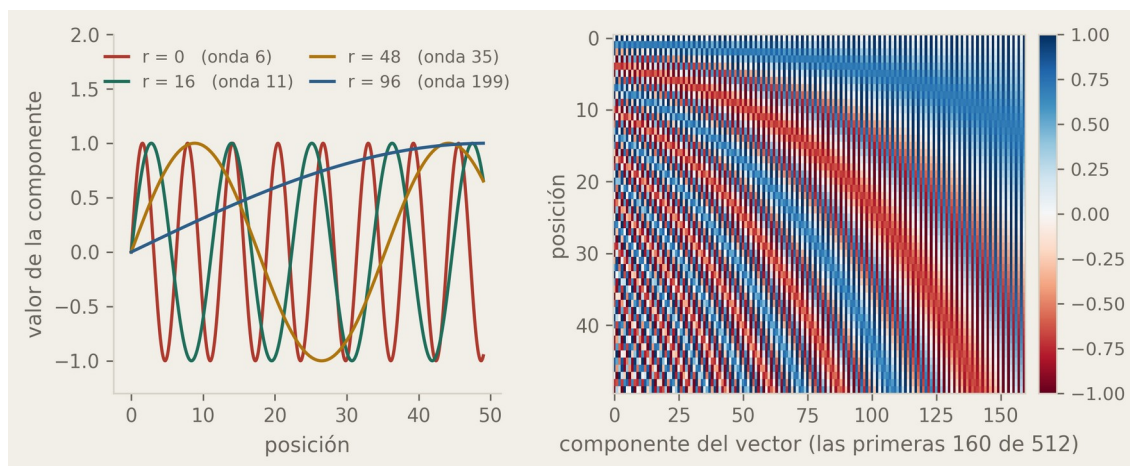
Positional encoding

La atención no sabe en qué orden vienen las piezas. A cada embedding se le suma un vector que depende solo de la posición, hecho con senos y cosenos.

$$\text{PE}(i, 2r) = \sin(i / 10000^{2r/d}) \quad i \text{ es la posición, } r \text{ la componente}$$

$$\text{PE}(i, 2r+1) = \cos(i / 10000^{2r/d}) \quad \text{ondas de } 2\pi \text{ a } 10000 \cdot 2\pi$$

$$x_i = e_i + \text{PE}(i) \quad \text{se suma al embedding, no se concatena}$$



Se suma, no se concatena

El vector de posición tiene la misma dimensión que el embedding.

No se aprende, se calcula

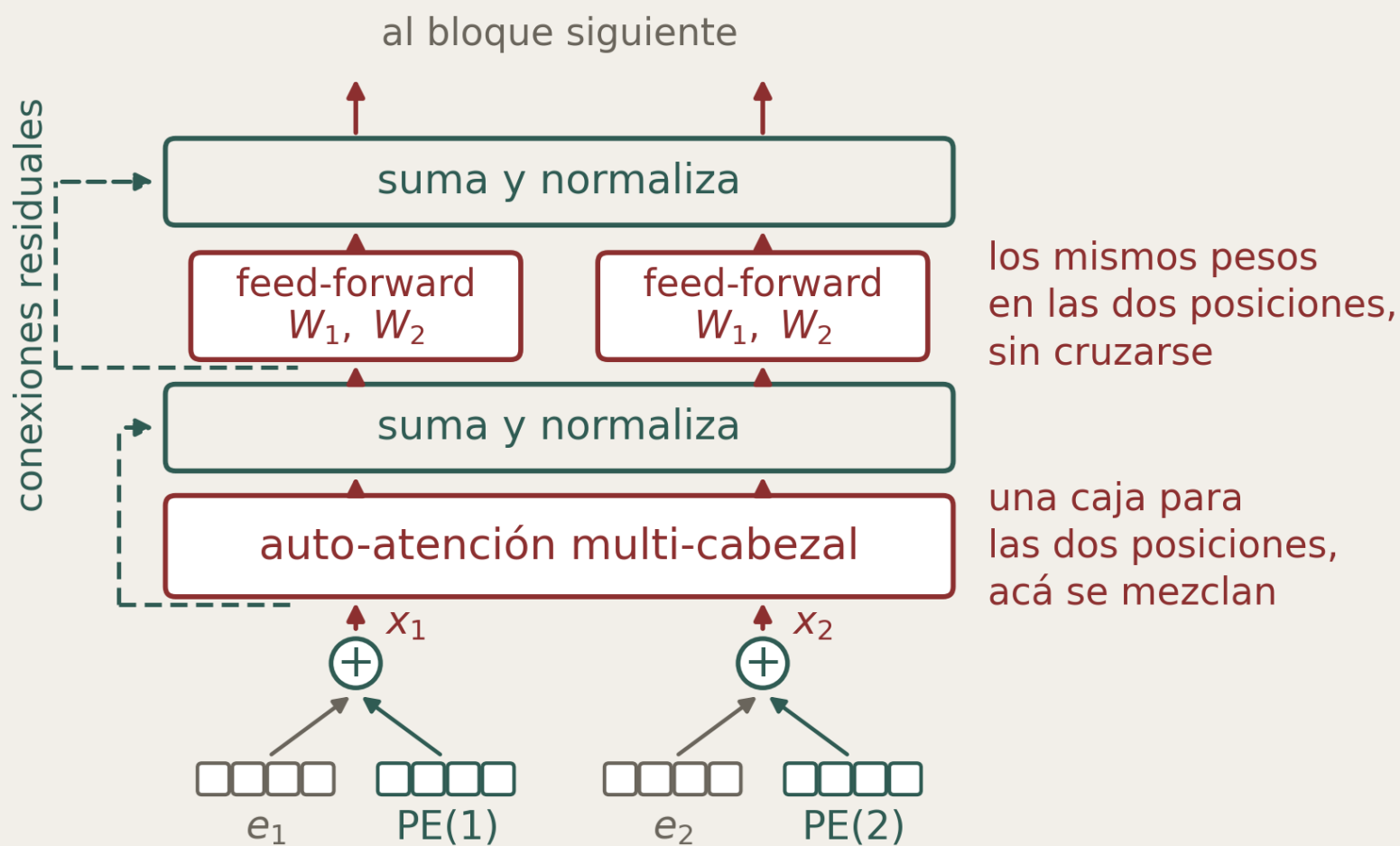
Sale de una fórmula fija, la misma para cualquier modelo y cualquier largo.

Cada componente es una senoide

Con ondas que van de 2π a 10.000 por 2π , de rapidísimas a lentísimas.

El bloque completo

Todo lo que fuimos agregando, en el orden en que se aplica. Un bloque entra y sale en la misma dimensión, así que encima va otro igual.



el vector de posición tiene el mismo largo, por eso se puede sumar

Atención global

La misma idea de la bidireccional de la clase 4, pero acá no hace falta una segunda red. Basta con dejar que la suma recorra todas las posiciones.

$$\text{head} = \text{softmax}\left(\text{mask}\left(\frac{QK^T}{\sqrt{d_k}}\right)\right) V$$

la máscara entra antes de la softmax

$$\text{mask}_{ij} = -\infty \text{ si } j > i$$

causal, la que veníamos usando

$$\text{head} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

global, sin máscara ninguna



y en modelos entrenados, la masa de atención por encima de la diagonal:

GPT-2, decoder 0,00 de 13

RoBERTa, encoder 4,74 de 11

el 43 % de la atención del encoder mira hacia adelante

Es sacar la restricción

Con j menor o igual que i cada posición mira atrás; sin ella, mira a todas.

En la BiLSTM costaba una red entera

Había que correr una segunda cadena al revés y concatenar.

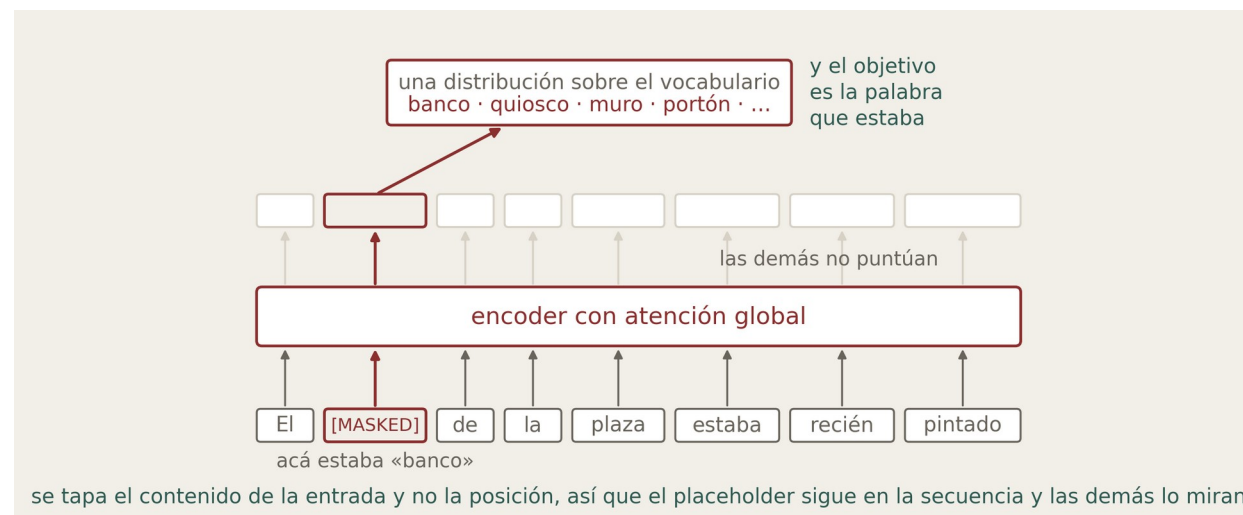
Masked language model

Se tapa una parte de la entrada y se le pide al modelo que la reconstruya.

$\tilde{x}_i = [\text{MASKED}]$ si $i \in \mathbf{M}$ $\mathbf{M}$ es una fracción de las posiciones

$h = \text{Encoder}(\tilde{x}_1, \dots, \tilde{x}_n)$ con atención global, sin restricción

$\mathcal{L} = - \sum_{i \in \mathbf{M}} \log P(x_i | \tilde{x})$ solo puntúan las posiciones tapadas



Se tapa el contenido, no la posición

El placeholder sigue en la secuencia y las demás lo pueden mirar.

La pérdida va solo en los placeholders

Las demás salidas se calculan igual, pero no puntúan.

BERT

Toma el esquema anterior y lo usa para preentrenar, con una segunda tarea colgada de la misma red.

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in M} \log P(x_i | \tilde{x})$$

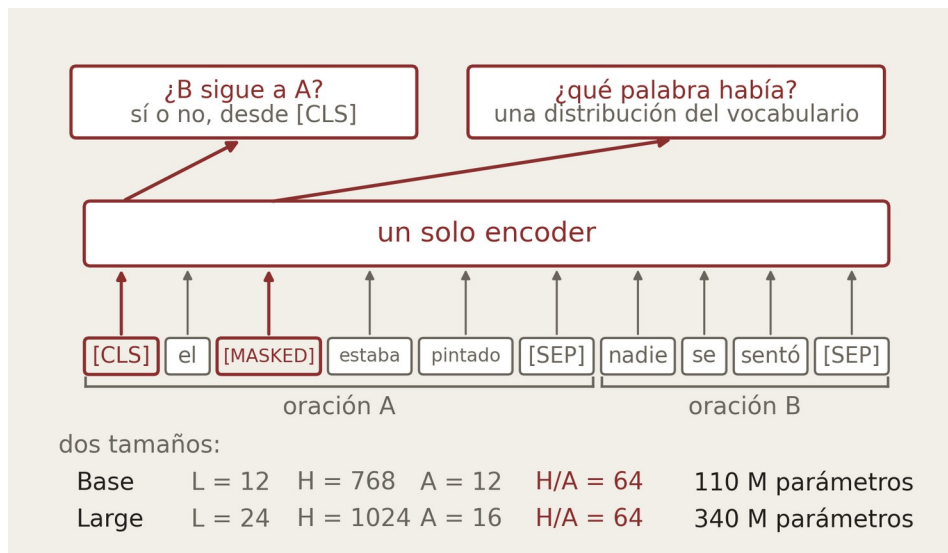
reconstruir lo enmascarado

$$\mathcal{L}_{\text{NSP}} = -\log P(y | h_{[\text{CLS}]}) , \quad y \in \{0, 1\}$$

¿la oración B sigue a la A?

$$\mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$$

se suman, y sin peso entre ellas



Dos tareas, un solo encoder

Reconstruir lo enmascarado y decidir si una oración sigue a la otra.

Las dos pérdidas se suman

Sin peso entre ellas, optimizadas a la vez sobre los mismos pesos.

H sobre A vuelve a dar 64

En los dos tamaños, igual que en el transformer original.

Devlin et al., «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding», NAACL 2019.

Qué sale del encoder

Un vector por posición, y todos dependen de la frase entera. Encima se le cuelga el cabezal que la tarea pida.

$$z_1, \dots, z_n = \text{Encoder}(x_1, \dots, x_n)$$

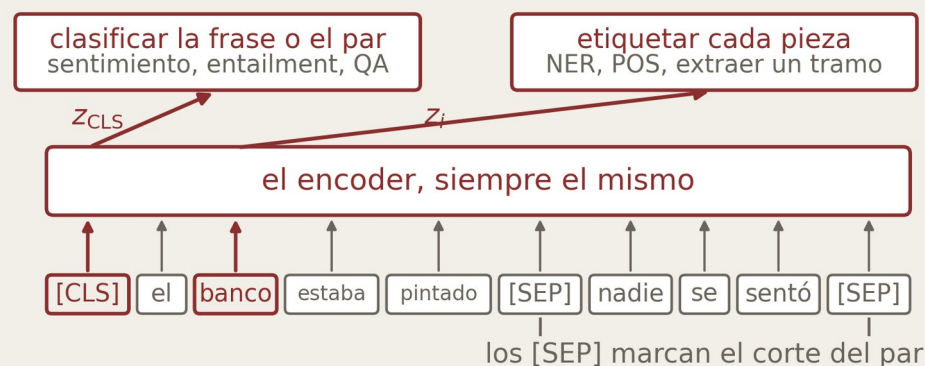
una salida por posición, de largo H

$$z_i = z_i(x_1, \dots, x_n)$$

depende de toda la frase, no solo de x_i

$$\hat{y} = f(z_{[\text{CLS}]}) \quad \text{o} \quad \hat{y}_i = g(z_i)$$

un cabezal distinto por tarea



la misma palabra en dos frases distintas

«el **banco** de la plaza estaba recién pintado»

«el **banco** me cobró una comisión muy alta»

a la salida del encoder los dos vectores de «banco» tienen coseno 0,657 y con «pintado», que en las dos frases significa lo mismo, da 0,945

El primero resume la frase

Sin contenido propio, pero mira a todas, así que sirve de embedding.

Los demás son la pieza en contexto

La misma palabra en dos frases distintas da dos vectores distintos.

El encoder no cambia entre tareas

Se preentrena una vez y arriba se le pone lo que haga falta.